

TaxDesk Deep Dive, From First Principles

This document teaches the whole project to a first time engineer. Not what the code does, but why it exists, how it works inside, and how to reason about it well enough to modify it alone.

How to use the three documents together

- this deep dive teaches the system top to bottom, read it in order
- the engineering journal tells the story of how we got here, entry by entry
- the code map at the bottom of the journal is the per file reference

Genuinely unknown things are marked "Unknown, needs dad". Nothing in this document is invented.

Part 1, The Problem and the Shape of the Solution

The business problem

Dad is a solo tax practitioner. Every month, each of his clients needs some of four government filings, GSTR-3B, GSTR-1, EPF, ESI. He tracks who still owes what in an Excel sheet with tick marks, by hand.

What happens without a better tool. Ticks get forgotten, the sheet and reality drift apart, and a missed filing means a government penalty with a client's name on it. The proof files, challans and acknowledgements, live in folders with no link to the ticks at all.

Unknown, needs dad. His exact client count, and the real due day for each filing type. The system runs without both today.

Why this exact solution

Three decisions define the whole product, and each came from the problem, not from fashion.

1. Local first. The data is dad's client information. It stays on his machine, in one file he owns. No cloud account, no monthly fee, no trust question. The trade-off is no access from a second device, which he does not need.
2. A browser app on localhost. He gets normal pages and buttons without us building an installer yet. The server and the browser run on the same machine.
3. Boring technology, deliberately. Python, SQLite, plain HTML. Chosen in ADR 001 because the second goal of this project is that its builder understands every layer. A framework that hides the SQL would defeat the point.

Why the architecture has layers at all

Start from zero. Why not just Excel with better formulas? Because the rules we care about, no duplicate tasks, no task without a client, cannot be enforced by a spreadsheet. Enforcement needs a database.

Why not let the pages talk to the database directly? Imagine the Dashboard template running SQL. Then a change to a table breaks HTML files, and a wrong query in a template can write data during what should be a read. Mixing jobs multiplies the damage any change can do.

So each job gets one home, and each home talks only to the next one down.

browser	
	HTTP, requests and responses
routes	app/routes/, web code, parses requests, redirects
	plain function calls
logic	app/services/ and app/db/queries.py, decisions and SQL
	SQL over one connection
SQLite	taxdesk.db, one file, owns the rules
disk	dad's client folders, never restructured by us

Delete test for each layer, what breaks if it disappears.

- remove routes, no way for a browser to reach the logic
- remove the queries layer, SQL spreads into every route, and the Dashboard and Priority pages can drift apart, our central guarantee
- remove the database, rules become suggestions, duplicates return
- remove the disk folders, nothing, they are dad's, we only read them

Part 2, The Concepts, In the Order You Need Them

Each concept follows the same path. The problem, the idea, how it works, the trade-off, and where it lives in TaxDesk.

2.1 HTTP, the request and response cycle

The problem. Two programs, browser and server, need a common language.

The idea. The browser sends a request, one line of intent plus details. The server sends back a response, a status code plus content. Nothing happens between requests, the connection is stateless, each request must carry everything needed to answer it.

The parts we actually use.

- methods, GET means read me a page, POST means save what I sent
- status codes, 200 page follows, 302 and 303 go elsewhere, 400 bad input, 404 no such thing, 500 the server crashed
- the body, form fields on the way in, HTML on the way out

In TaxDesk. Every function in `app/routes/` answers exactly one method plus path pair. Reads are GET, writes are POST, never mixed, because browsers repeat GETs freely (refresh, back button) and a repeated write would corrupt data.

The 303 trick, post redirect get. After every successful POST we respond with 303, which orders the browser to GET another page. So refresh always repeats the harmless GET, never the write. Look at any route in `app/routes/periods.py`, every POST ends in `RedirectResponse` with `status_code 303`.

2.2 Routing, how a URL finds a function

The problem. One server, many pages, requests must reach the right code.

The idea. A table from method plus path to function. FastAPI builds this table from decorators, `@router.get("/dashboard")` above a function registers it.

In TaxDesk. `app/main.py` assembles the app and plugs in four route modules with `include_router`. When a request arrives, FastAPI matches the path, pulls typed values out of it (`client_id` in `/clients/{client_id}` arrives as an int), and calls the function.

2.3 Templates, and the attack they prevent

The problem. Pages need real data inside HTML, and building HTML by gluing strings invites disaster. If a client is named `<script>steal()</script>` and we glue that into the page, the browser runs it. That attack is called XSS, cross site scripting.

The idea. A template is HTML with typed blanks. The engine, Jinja2, fills each blank and escapes the value on the way in, `<` becomes `<`, so data can never become code.

In TaxDesk. `app/templates/`, `base.html` is the shared frame, each page extends it. Routes pass a plain dict of values. Templates never call our code, they only display. Escaping is on by default, we did not have to remember it, which is exactly why we chose an engine over string gluing.

2.4 The relational database, from zero

The problem. The product's core promise is a set of facts with rules, every task belongs to a real client, the same task can never exist twice. Facts with enforced rules is precisely what a relational database is.

The idea. Data lives in tables, each row one fact, each column one attribute. Rows point at each other by id numbers. Rules are declared once, on the table, and the engine refuses every write that breaks them, no matter which code attempts it.

Why SQLite specifically. It is a library, not a server, the whole database is one file, taxdesk.db. For one office on one machine that is a perfect fit, and its limitation, poor fit for many simultaneous writers, is a problem we do not have.

In TaxDesk, six tables, created by app/db/migrations/001_initial_schema.sql.

clients	who dad works for, plus their folder on disk
client_services	which of the 4 filings each client needs
compliance_periods	one row per tracked month
compliance_tasks	the heart, one row = client x service x month
documents	saved file links for search, arrives in v0.6
settings	key value facts, first row is the root folder

2.5 Keys and relationships

A primary key is a row's permanent identity, we use plain integer ids. Why not use the client's name? Names get edited, ids never do, so everything else can point at the id safely.

A foreign key is a column holding another table's id. `compliance_tasks.client_id` points at `clients.id`, which makes a task belong to a client. The engine refuses a task pointing at an id that does not exist, so orphan rows are impossible.

One SQLite trap worth remembering forever. Foreign key enforcement is OFF per connection unless you turn it on. Our `connect()` in `app/db/migrate.py` runs `PRAGMA foreign_keys = ON` every time, and it is the only sanctioned way to open the database in this codebase.

2.6 Constraints, rules the code cannot break

The problem. A rule enforced only in Python holds until one forgotten code path breaks it silently.

The idea. Declare rules on the table, the engine checks every write from anyone forever.

The one that carries the product, on `compliance_tasks`.

```
UNIQUE (client_id, period_id, service_type)
```

One client, one service, one month, at most one task. Generation can run twice, a button can be double clicked, a bug can retry, the database refuses the duplicate every time. We proved this with real inserts on day one, and a test guards it still.

Also used, CHECK constraints as lightweight enums, `status IN ('pending', 'done', 'not_applicable')` rejects typos at write time, and NOT NULL forbids missing values.

2.7 SQL, the four statements we live on

SELECT reads rows, INSERT adds, UPDATE changes, and two combinations do our heavy lifting.

JOIN, from first principles. Tables stand alone. A task row stores `client_id` 7, a number. To show "Kumar Textiles" the query says `JOIN clients c ON c.id = t.client_id`, meaning for each task, find the client row whose id matches and glue the rows together. Matching rows across tables by a shared key, that is the entire concept.

GROUP BY. Rows in, buckets out. `GROUP BY service_type` with `COUNT(*)` collapses many task rows into one count per service. The Dashboard numbers are exactly this query, computed fresh on every glance, never stored. A stored count can drift from the truth, a computed one cannot, and that single principle is why Dashboard and Priority can never disagree.

Parameterized queries. Every value goes in through `?` placeholders, never glued into the SQL string. Gluing user input into SQL is the SQL injection attack, the database cannot tell data from command. The placeholder keeps them separate at the protocol level. Check any function in `app/db/queries.py`, there is no string glued SQL anywhere.

2.8 Transactions, all or nothing

The problem. Some changes come in pairs that must not be separable. The migration runner executes a schema file AND records it in the logbook. If the process died between the two, the database would lie about itself.

The idea. A transaction groups writes, commit makes them all permanent together, and a crash before commit means none of them happened. This is the A in ACID, atomicity, with consistency, isolation, and durability as its siblings.

In TaxDesk, two places to see it.

- `app/db/migrate.py`, execute file, record filename, then commit, one unit
- `app/deps.py get_db()`, each web request gets one connection, commit happens only if the route finished without an exception, so a failed request writes nothing at all

2.9 Migrations, how structure travels

The problem. `taxdesk.db` never leaves dad's machine, it is his data. But the table structure must reach every machine identically, yours, his, every test.

The idea. Structure changes are numbered SQL files in git. A runner applies the not yet applied ones in order and records each in a logbook table inside the database itself, so every database knows its own state. Applied files are frozen forever, change means a new file.

In TaxDesk, `app/db/migrate.py` is the runner, about fifty lines, and the whole algorithm is one set difference, files on disk minus files in the logbook. Startup runs it automatically, so dad installing an update just restarts the app.

2.10 Idempotency, the property that forgives repetition

Definition. An operation is idempotent when running it twice leaves the same result as running it once.

Why it matters here. Real usage is messy, buttons get double clicked, scripts get rerun, people forget what they already did. Every repeatable operation in TaxDesk was built idempotent on purpose.

- generation, INSERT OR IGNORE plus the UNIQUE constraint, second run inserts zero
- migrations, the logbook makes reruns a no-op
- seeding, safe to run any number of times
- form posts, the 303 redirect stops browsers from repeating them

2.11 Dependency injection, the small kind

The problem. Every route needs a database connection, opened and closed correctly, every time.

The idea. Instead of each route creating its own, the route declares what it needs and the framework hands it in. That is all dependency injection means, receive your tools, do not build them.

In TaxDesk. A route's signature says `conn: Connection = Depends(get_db)`. FastAPI sees it, calls `get_db()` in `app/deps.py`, which opens a connection through `connect()`, yields it to the route, commits if the route succeeded, closes always. Twenty routes, one correct connection lifecycle, written once.

2.12 Async and sync, what the two function kinds mean

Synchronous code holds its thread until done. Asynchronous code can pause at an `await` and let the server do other work while waiting.

Our rule, async only where something genuinely awaits. The three POST routes that read forms are async because `await request.form()` waits for the body to arrive over the network. Every other route is a plain `def`, and FastAPI runs those on a thread pool automatically.

The scar this left. FastAPI may open a connection on one thread and use it on another within a request, and SQLite forbids cross thread use by default. Our first test run crashed exactly there, and the fix is the documented pattern, `check_same_thread=False` in `connect()`, safe because a connection never leaves its single request.

2.13 Testing, machine checked promises

The problem. "It worked when I tried it" decays. Code changes, and yesterday's behavior silently breaks.

The idea. A test is a small program that exercises the real code and asserts the result. It fails loudly the moment a promise breaks. A fixture is shared setup a test asks for by name.

In TaxDesk, `tests/conftest.py` builds every test a fresh, fully migrated database in a temp folder, so tests are fast, isolated, and never touch real data. The question worth asking of every test is what bug would it catch. Three examples.

- `test_second_generation_run_creates_nothing` catches any future change that breaks duplicate protection

- `test_dashboard_and_priority_always_agree` catches the two pages drifting apart, our core product requirement, machine enforced
 - `test_back_to_pending_clears_completion_trace` catches stale completion data lying about history
-

Part 3, Tracing the Code, From Click to Row and Back

3.1 What happens when the app starts

```
you run, venv/bin/uvicorn app.main:app
1. uvicorn starts and imports app/main.py
2. importing main imports the route modules, their tables of
   method+path to function get registered
3. lifespan() runs once, connect() opens or creates taxdesk.db,
   migrate() applies any pending schema files
4. uvicorn opens the port and waits for requests
```

3.2 A read, GET /dashboard, end to end

```
1. browser sends GET /dashboard
2. uvicorn hands the request to FastAPI
3. FastAPI matches the path, target is dashboard() in
   app/routes/dashboard.py
4. the signature asks for a connection, get_db() opens one
5. pick_period() decides which month, the URL first, else
   default_period() picks the newest open one
6. queries.pending_counts() runs the GROUP BY, queries
   .pending_clients() runs the JOIN, plain rows come back
7. the route hands rows to dashboard.html, Jinja2 fills and escapes
8. response 200 travels back, get_db() commits (nothing to commit on
   a read) and closes the connection
```

Under the hood at step 6, SQLite parses the SQL, plans how to find the rows, scans the task table filtering by period and status, groups, counts, and returns. With no indexes it reads the whole table, which at our scale, a few thousand rows per year, costs under a millisecond.

3.3 A write, POST /periods/5/generate, end to end

1. browser sends POST /periods/5/generate
2. FastAPI calls generate() in app/routes/periods.py with period_id 5 already parsed to int
3. the route loads the period, missing means 404
4. closed period, redirect back with error=closed, nothing written
5. open, so generate_tasks() in app/services/generation.py runs the one INSERT OR IGNORE SELECT, the database inserts every missing client x service row and silently skips existing ones, the UNIQUE constraint is the gatekeeper
6. due dates backfill where rules exist, today none do
7. route responds 303 to /periods/5
8. get_db() commits, the new rows become permanent
9. the browser GETs the period page and shows the fresh tasks

The line worth reading twice, in generation.py.

```
cursor = conn.execute(
    "INSERT OR IGNORE INTO compliance_tasks (client_id, period_id, service_type)"
    " SELECT client_id, ?, service_type FROM client_services WHERE active = 1",
    (period_id,),
)
```

Level 1, plain English. Copy every active client service pair into the tasks table for this month, skipping pairs already there.

Level 2, engineering. One set based statement instead of a Python loop, the database does the work where the data lives, and OR IGNORE plus UNIQUE makes it idempotent.

Level 3, under the hood. SQLite runs the inner SELECT, and for each row attempts an insert, checks the UNIQUE index, inserts or skips. cursor.rowcount afterwards counts only real inserts, which is how the caller can report 13 new, then 0 new.

3.4 The trust boundary, onboarding confirm

Form input is untrusted even on localhost. confirm_clients() in app/routes/onboarding.py recomputes the real subfolders of the root and creates clients only for submitted names inside that set, so a crafted name like ../evil is silently dropped. The principle, validate input where it crosses into your system, using what the system itself knows to be true.

3.5 If we deleted it, what breaks

- connect(), foreign keys quietly stop being enforced everywhere, orphan rows appear weeks later
- get_db(), every route hand rolls connections, one will forget to close or commit
- the UNIQUE constraint, duplicate tasks return on the first double click, and the Dashboard counts them
- the 303 redirects, browser refresh resubmits forms, double writes

- queries.py, SQL scatters into routes, the mirror guarantee now depends on copy pasted WHERE clauses staying in sync by luck
 - the logbook table, migrations rerun on every start and crash on existing tables
-

Part 4, The Database, One Level Deeper

Why no indexes yet, and how to think about it

An index is a sorted lookup structure that lets the database jump to matching rows instead of scanning everything. It costs disk space and a little work on every write.

The reasoning process, always in this order. Which query is slow? None. How much data does a scan touch? Forty clients times four services times twelve months is under two thousand rows a year. When would that change? Years of data, hundreds of clients, or the documents table after the scanner fills it. Then the first index would go on compliance_tasks (period_id, status), because every glance filters on exactly that pair. Adding it today would be optimizing a problem we measured to not exist.

Concurrency, honestly

One user, one process. SQLite locks the file per write, so even a double click cannot interleave two half done writes. The one real race we could have, generate clicked twice fast, is absorbed by the UNIQUE constraint, not by locks. Distributed system concerns, replication, sharding, eventual consistency, genuinely do not apply to one office on one machine, and pretending otherwise would be decoration.

ACID mapped to our code

- atomicity, get_db() commits all of a request's writes or none
 - consistency, constraints keep every committed state legal
 - isolation, single writer at a time via SQLite's file lock
 - durability, committed data survives a crash, SQLite journals writes
-

Part 5, What We Deliberately Do Not Have

Each entry, what it is, why systems need it, why we do not, and what would change the answer.

- authentication and authorization. Proving who you are, and what you may do. The app binds to localhost on dad's own machine, the operating system login IS the front door. Trigger to revisit, the moment the app is reachable from any other device.

- caching. Saving computed results to skip recomputing. Our heaviest query costs under a millisecond, and a cache would introduce the stale count problem we designed out. Trigger, measured slowness.
- background jobs and queues. For work too slow for a request. Nothing we do is slow yet. The v0.6 folder scan runs on demand, and only if real scans measure slow does a job runner earn a place.
- an ORM. A library that hides SQL behind objects. Rejected in ADR 001 because visible SQL is a learning goal here, and our query layer is small. Trade-off accepted, we write a little more by hand.
- CSRF protection. A real gap, not a virtue. Another website open in the same browser could auto submit a form to localhost. Impact is low today, single user, no auth to steal, but this gets fixed before the app ever leaves localhost. Named in the debt list.
- CI, containers, cloud. No pipeline runs our tests automatically yet, a genuine recommended improvement, one GitHub Action running pytest on every PR. Containers and cloud solve deployment repeatability and scale we do not have.

Part 6, Failure Modes

failure	detection	handling and recovery
bad form input written	route validation	400 or error redirect, nothing
month 13, status typo	CHECK constraint	write refused by the database itself
task for missing client	FOREIGN KEY	write refused
double generate new	UNIQUE	second insert ignored, count says 0
refresh after submit	303 redirect	browser repeats a harmless GET
write to closed month	route status check	refused with visible message
app crash mid request consistent	no commit happened	SQLite journal, database stays
db file missing only via backup	connect() creates	migrations rebuild structure, data
client folder renamed	v0.6 scan will flag	folder_path reported, dad relinks
disk dies	nothing, today	NO BACKUP EXISTS, the top item of
Part 7		

Part 7, Senior Review of Our Own System

Facts first, then recommendations, kept separate.

What is genuinely good

- rules live in the schema, the UNIQUE and CHECK constraints make the worst data bugs impossible rather than unlikely

- one query layer, and the mirror requirement is enforced by a test, not a promise
- everything repeatable is idempotent, generation, migrations, seed
- the documentation system, journal, code map, docstrings with ins and outs, is unusually strong for a project this size

What concerns me, in priority order

1. No backup of taxdesk.db. Dad's real months of work will live in one file on one disk. Problem, disk death loses everything. Better, on every app start copy the db to a dated file in a backups folder, keep the last N. Cheap, boring, urgent before the v0.7 trial.
2. No CI. Tests exist but only run when remembered. A GitHub Action running pytest on every PR closes the gap for near zero cost.
3. CSRF, as described in Part 5. Fix before any network exposure.
4. The service list lives in two places, the CHECK constraints in the schema and SERVICE_TYPES in queries.py. Adding a fifth service means a migration AND a code edit, and forgetting one produces confusing failures. Acceptable at four services, worth a comment in both places pointing at each other.
5. clients.updated_at is written once and never updated, SQLite has no automatic mechanism. Either maintain it on every client UPDATE when editing arrives, or drop it in a future migration. Dormant, not dangerous.

What I would not change

Raw SQL, no ORM. Localhost only for now. No caching, no queues, no scaling machinery. Each absence is measured against a real workload of one office, and each has a named trigger for revisiting.

Part 8, Things Not to Confuse

- SQLite vs SQL vs database. SQL is the language, a database is the organized data, SQLite is the engine we use to run one.
- FastAPI vs uvicorn. FastAPI decides what to answer, uvicorn is the server that listens on the port and hands requests over.
- GET vs POST. Read vs write. Browsers repeat GETs freely, so writes must never hide behind them.
- 302 vs 303. Both redirect, 303 additionally forces the next request to be a GET, which is what makes post redirect get work.
- commit in git vs commit in a database. A saved code snapshot vs making a transaction's writes permanent. Same word, unrelated.
- schema vs data. The cabinet vs its contents. Migrations ship the cabinet, the contents never leave dad's machine.

- authentication vs authorization. Who you are vs what you may do. We currently have neither, by explicit decision, see Part 5.
 - async vs threaded. Our async routes pause at await on the event loop, our sync routes run on a thread pool. Both serve one user fine, the distinction mattered once, in the check_same_thread bug.
-

Part 9, Questions I Should Be Able to Answer

Try each from memory first. Every answer is in this document, the journal teaches the fundamentals behind them.

Product and architecture.

1. What breaks for dad if this project does not exist?
2. Why three layers, and what breaks when each is removed?
3. Why is the database file gitignored while migrations are committed?

Code and flow.

1. Trace a click on Generate from browser to database and back, naming the files involved.
2. Where does a browser request first touch our code?
3. Which single function makes foreign keys work everywhere, and why is it the only allowed door to the database?

Data.

1. Why can a duplicate task never exist, even if the code is buggy?
2. Why are the Dashboard numbers computed on every request instead of stored?
3. When would we add our first index, on which columns, and why not now?

Change.

1. To add a fifth service type, which two places must change, and what happens if you forget one?
 2. If dad asks for a Notes box on the client page, which files change?
 3. What is the first thing to build before dad's real trial, and why is it not a feature?
-

Part 10, Learning Roadmap

Level 1, must understand, everything in this document. HTTP and the request cycle, routing, templates and escaping, tables, keys, constraints, JOIN and GROUP BY, transactions, migrations, idempotency, layering, tests and fixtures. Each appears in a file you own.

Level 2, should understand next. Indexes and query plans (EXPLAIN QUERY PLAN in SQLite, try it on pending_counts), CSRF and the OWASP basics, CI pipelines (a pytest GitHub Action would be your first), packaging Python apps for a machine you do not control, and SQLite's ALTER TABLE limits before migration 002 needs them.

Level 3, advanced, learn when the trigger fires, not before. Concurrency beyond one user, real authentication and sessions, caching and its invalidation problem, and the distributed systems canon, replication, partitioning, consensus. TaxDesk needs none of it today, and knowing WHY it does not is itself the system design skill.

Part 11, The Final Mental Model

```
a missed filing costs dad money and trust
    |
so track four filings per client per month, locally, simply
    |
six tables hold the facts, constraints make bad facts impossible
    |
three layers, routes parse, logic decides, SQLite enforces
    |
every page is a query over compliance_tasks, computed fresh
    |
every repeatable action is idempotent, so mistakes are harmless
    |
migrations move structure between machines, data never moves
    |
tests lock the promises, the journal and this document keep
the understanding
```

The modification drill, proof of understanding. Dad asks for a fifth service, Professional Tax. The changes, in order. A new migration 002 widening the CHECK constraints (SQLite requires the rebuild pattern, new table, copy, drop, rename), the SERVICE_TYPES list and label in app/db/queries.py, and nothing else, generation, pages, and tests pick it up from those two sources. If you can explain WHY nothing else changes, you understand the system.